

▼ NLTK

Skup biblioteka i programa pisanih U Pythonu sa svrhom obrade prirodnog jezika

link:

<https://www.nltk.org/>

NLTK punkt_tab je podatkovni paket koji koristi statističke modele kako bi identificirao krajeve rečenica, ne koristeći samo znakove točke i sl.

punkt_tab je novija, sigurnija verzija paketa koja je zamijenila standardni punkt paket

Primjeri: punkt_tab zna da Mr. ili U.S. ne označavaju kraj rečenice. `from nltk.tokenize import sent_tokenize` vs. `nltk.download("punkt_tab")`

Prvo uvozi Python funkciju, a drugo uvozi unaprijed trenirani model spremljen kao podatkovnu datoteku

```
import nltk
from nltk.tokenize import word_tokenize
nltk.download("punkt_tab")
from nltk.corpus import stopwords
nltk.download("stopwords")
engleski_stop_words = set(stopwords.words("english"))
from nltk.stem import PorterStemmer
from nltk.stem import WordNetLemmatizer
nltk.download('wordnet')
```

```
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data] Unzipping tokenizers/punkt_tab.zip.
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data] Unzipping corpora/stopwords.zip.
[nltk_data] Downloading package wordnet to /root/nltk_data...
True
```

```
corpus = """Hello and welcome!
I hope you will have fun during our NLP training!
If you have any questions, feel free to ask for help.
"""
```

```
print (corpus)
```

```
Hello and welcome!
I hope you will have fun during our NLP training!
If you have any questions, feel free to ask for help.
```

1. KORIŠTENJE PYTHON SPLIT() METODE

▼ Koji imamo problem ako koristimo metodu split()?

```
splitani_corpus = corpus.split()
```

```
print (splitani_corpus)
```

```
['Hello', 'and', 'welcome!', 'I', 'hope', 'you', 'will', 'have', 'fun', 'during', 'our', 'NLP', 'training!', 'If', 'you', 'h
```

▼ 2. TOKENIZACIJA

Tokenizacija u obradi prirodnog jezika (NLP) je temeljni proces rastavljanja teksta (rečenica, odlomaka ili cijelih dokumenata) na manje, smislene jedinice koje se nazivaju tokeni

Tokenizacija je nužan prvi korak jer omogućuje računalnim modelima da "razumiju" ljudski jezik pretvarajući nestrukturirani tekst u strukturirane podatke, koji se zatim mogu pretvoriti u brojčane reprezentacije (npr. ID-

eve) pogodne za strojno učenje.

✓ Podjela teksta na tokene

```
tokens = word_tokenize(corpus)
```

```
print (tokens)
```

```
['Hello', 'and', 'welcome', '!', 'I', 'hope', 'you', 'will', 'have', 'fun', 'during', 'our', 'NLP', 'training', '!', 'If', 'I', 'can', 'help', 'you', 'with', 'your', 'project']
```

Čišćenje podataka - lowercasing i interpunkcijski znakovi

✓ Lowecasing je normalizacija, smanjujemo dimenzionalnost (broj tokena)

Ne zanima nas razlika između Movie i movie

```
finalni_tokeni = []
```

```
for token in tokens:
```

```
    token_mala_slova = token.lower()
```

```
    if token_mala_slova.isalnum():
```

```
        finalni_tokeni.append(token_mala_slova)
```

```
print(finalni_tokeni)
```

```
['hello', 'and', 'welcome', 'i', 'hope', 'you', 'will', 'have', 'fun', 'during', 'our', 'nlp', 'training', 'if', 'you', 'can', 'help', 'you', 'with', 'your', 'project']
```

✓ 3. MICANJE STOP RIJEČI

✓ Micanje stop riječi

Stop riječi su uobičajene riječi u tekstu koje mičemo jer nose vrlo malo ili nimalo jedinstvenog značenja koje bi pomoglo u razumijevanju teme teksta. U fazi pretprocesiranja teksta, ove se riječi obično uklanjaju kako bi se smanjila dimenzionalnost podataka i povećala učinkovitost modela

```
stopwords.words("english")
```

```
['a',  
'about',  
'above',  
'after',  
'again',  
'against',  
'ain',  
'all',  
'am',  
'an',  
'and',  
'any',  
'are',  
'aren',  
'aren't',  
'as',  
'at',  
'be',  
'because',  
'been',  
'before',  
'being',  
'below',  
'between',  
'both',  
'but',  
'by',  
'can',  
'couldn',  
'couldn't',  
'd',  
'did',  
'didn',  
'didn't',  
'e',  
'each',  
'either',  
'enough',  
'even',  
'ever',  
'few',  
'for',  
'from',  
'further',  
'happened',  
'has',  
'hasn',  
'hasn't',  
'he',  
'he'd',  
'he's',  
'her',  
'here',  
'hers',  
'herself',  
'him',  
'himself',  
'his',  
'how',  
'however',  
'i',  
'i'd',  
'i.e.',  
'if',  
'in',  
'into',  
'is',  
'isn',  
'isn't',  
'it',  
'it'd',  
'it's',  
'me',  
'more',  
'most',  
'much',  
'my',  
'myself',  
'n',  
'ne',  
'never',  
'no',  
'nor',  
'not',  
'of',  
'off',  
'on',  
'once',  
'only',  
'or',  
'other',  
'others',  
'out',  
'over',  
'own',  
'per',  
'perhaps',  
're',  
's',  
'she',  
'she'd',  
'she's',  
'so',  
'some',  
'somebody',  
'someone',  
'something',  
'such',  
'such as',  
'than',  
'that',  
'that'd',  
'that's',  
'the',  
'there',  
'there'd',  
'there's',  
'they',  
'they'd',  
'they're',  
'this',  
'this'd',  
'this's',  
'those',  
'through',  
'to',  
'too',  
'towards',  
'two',  
'us',  
'us'd',  
'us's',  
'v',  
've',  
'very',  
'was',  
'wasn',  
'wasn't',  
'we',  
'we'd',  
'we're',  
'well',  
'when',  
'where',  
'where'd',  
'where's',  
'which',  
'while',  
'who',  
'who'd',  
'who's',  
'with',  
'without',  
'would',  
'wouldn',  
'wouldn't',  
'x',  
'y',  
'you',  
'you'd',  
'you're',  
'your',  
'yourself',  
'yourself's',  
'z',  
'z's']
```

```
'do',
'does',
'doesn',
'doesn't',
'doing',
'don',
'don't',
'down',
'during',
'each',
'few',
'for',
'from',
'further',
'had',
'hadn',
'hadn't',
'has',
'hasn',
'hasn't',
'have',
'haven',
'haven't',
```

```
tokeni_bez_stop_word = []

for token in finalni_tokeni:
    if token not in engleski_stop_words:
        tokeni_bez_stop_word.append(token)

print (finalni_tokeni)
print (tokeni_bez_stop_word)
```

```
['hello', 'and', 'welcome', 'i', 'hope', 'you', 'will', 'have', 'fun', 'during', 'our', 'nlp', 'training', 'if', 'you', 'hav
['hello', 'welcome', 'hope', 'fun', 'nlp', 'training', 'questions', 'feel', 'free', 'ask', 'help']
```

Koje riječi su maknute?

Je li se izgubilo značenje?

Ako je problem jednostavan i podaci su ograničeni, klasični pristup često je bolji izbor. Ako je problem jezično složen i kontekst je ključan, tada dolaze napredni modeli.

Kada je klasični ML (BoW + Logistic Regression / SVM) sasvim dovoljan?

- Kratki tekstovi
- Jasna klasifikacija (spam / not spam)
- Ograničen budžet
- Mali dataset
- Brzina i jednostavnost važniji od 2–3% bolje točnosti
- Jasna klasifikacija (spam / not spam)
- Ograničen budžet
- Mali dataset
- Brzina i jednostavnost važniji od 2–3% bolje točnosti

Primjeri:

- spam filter
- klasifikacija ticketa
- jednostavan sentiment
- klasifikacija tema

▼ Zadaci

Na tekstovima corpus2 i corpus3 napravite:

- tokenizaciju
- lowercasing
- maknite interpunkcijske znakove

- maknite stop riječi

```
corpus2 = """The training session was clear and well organized.
The instructor explained each concept step by step.
Everyone understood the examples and asked relevant questions.
The atmosphere was positive and productive.
"""
```

```
print (corpus2)
```

```
The training session was clear and well organized.
The instructor explained each concept step by step.
Everyone understood the examples and asked relevant questions.
The atmosphere was positive and productive.
```

```
corpus3 = """I don't think Apple handled the update well.
The new design is not what I expected.
I bought an apple yesterday, and it worked better than my phone.
Great, another improvement that doesn't actually improve anything.
"""
```

```
print (corpus3)
```

```
I don't think Apple handled the update well.
The new design is not what I expected.
I bought an apple yesterday, and it worked better than my phone.
Great, another improvement that doesn't actually improve anything.
```

▼ Rješenje za corpus3

```
tokens3 = word_tokenize(corpus3)
```

```
finalni_tokeni3 = []

for token in tokens3:
    token_mala_slova = token.lower()

    if token_mala_slova.isalnum():
        finalni_tokeni3.append(token_mala_slova)

print(finalni_tokeni3)
```

```
['i', 'don', 't', 'think', 'apple', 'handled', 'the', 'update', 'well', 'the', 'new', 'design', 'is', 'not', 'what', 'i', 'e
```

```
tokeni_bez_stop_word3 = []

for token in finalni_tokeni3:
    if token not in engleski_stop_words:
        tokeni_bez_stop_word3.append(token)

print (finalni_tokeni3)
```

```
['i', 'don', 't', 'think', 'apple', 'handled', 'the', 'update', 'well', 'the', 'new', 'design', 'is', 'not', 'what', 'i', 'e
```

```
print (tokeni_bez_stop_word3)
```

```
['think', 'apple', 'handled', 'update', 'well', 'new', 'design', 'expected', 'bought', 'apple', 'yesterday', 'worked', 'bett
```

▼ Primjer

Koja bi bila svrha ovog (pseudo)koda?

```
spam_words = ["free", "winner", "money", "offer", "buy", "v i a g r a", "lottery"]

treshold = 3
spam = False

spam_score = 0
```

```

for token in tokens:
    if token in spam_words:
        spam_score += 1

if spam_score > threshold:
    spam = True

print (spam_score)
print (spam)
print (tokens)

```

```

1
False
['Hello', 'and', 'welcome', '!', 'I', 'hope', 'you', 'will', 'have', 'fun', 'during', 'our', 'NLP', 'training', '!', 'If', '

```

U ranim NLP sustavima tokenizacija je služila da razbijemo tekst na riječi i tražimo obrasce poput spam riječi. Kasnije smo počeli koristiti statističke modele poput Naive Bayesa, a danas neuralne mreže i transformere koji razumiju kontekst.

4. STEMMING I LEMATIZACIJA

Kad radimo NLP, isti oblik riječi se često javlja u više oblika primjeri:

- run
- running
- runs
- ran

Za model su to četiri različite riječi, ali semantički znače isto. Raste dimenzionalnost. Cilj je smanjiti kompleksnost modela i prebaciti sve riječi na isti osnovni oblik.

Stemming

Jednostavno rezanje riječi. Algoritam uklanja nastavke, ne razumije gramatiku

- running --> run
- runs --> run
- study --> stud
- studied --> stud

Kao rezultat imamo brz i jednostavan algoritam, pogodno za velike korpuse, ali gramatički neispravne riječi. Važno za naglasiti, ne koristi riječnike.

```

corpus4 = """
Students were studying different subjects.
Some students study every day.
Others studied only before exams.
"""

```

```
print (corpus4)
```

```

Students were studying different subjects.
Some students study every day.
Others studied only before exams.

```

```

tokens = word_tokenize(corpus4)

finalni_tokeni4 = []

for token in tokens:
    token = token.lower()

    if token.isalnum():
        finalni_tokeni4.append(token)

print(finalni_tokeni4)

```

```
['students', 'were', 'studying', 'different', 'subjects', 'some', 'students', 'study', 'every', 'day', 'others', 'studied',
```

```
tokeni_bez_stop_word4 = []
```

```
for token in finalni_tokeni4:
    if token not in engleski_stop_words:
        tokeni_bez_stop_word4.append(token)

print (tokeni_bez_stop_word4)
```

```
['students', 'studying', 'different', 'subjects', 'students', 'study', 'every', 'day', 'others', 'studied', 'exams']
```

```
stemmer = PorterStemmer()

stem_tokeni = []

for token in tokeni_bez_stop_word4:
    stem = stemmer.stem(token)
    stem_tokeni.append(stem)

print(stem_tokeni)
```

```
['student', 'studi', 'differ', 'subject', 'student', 'studi', 'everi', 'day', 'other', 'studi', 'exam']
```

▼ Lematizacija

Lingvistički pristup u kome model koristi rječnik i gramatička pravila

- running --> run
- better --> good
- cars --> car

Model je točniji, smisleniji, ali zahtjeva dodatne resurse i sporiji je.

Za razliku od stemminga gdje kao rezultat možemo dobiti riječi bez smisla, rezultat lematizacije su smislene riječi

```
corpus4 = """
Students were studying different subjects.
Some students study every day.
Others studied only before exams.
"""
```

```
tokens = word_tokenize(corpus4)

finalni_tokeni4 = []

for token in tokens:
    token = token.lower()

    if token.isalnum():
        finalni_tokeni4.append(token)

print(finalni_tokeni4)
```

```
['students', 'were', 'studying', 'different', 'subjects', 'some', 'students', 'study', 'every', 'day', 'others', 'studied',
```

```
tokeni_bez_stop_word4 = []

for token in finalni_tokeni4:
    if token not in engleski_stop_words:
        tokeni_bez_stop_word4.append(token)

print (tokeni_bez_stop_word4)
```

```
['students', 'studying', 'different', 'subjects', 'students', 'study', 'every', 'day', 'others', 'studied', 'exams']
```

```
lemmatizer = WordNetLemmatizer()

lemma_tokeni = []

for token in tokeni_bez_stop_word4:
    lemma = lemmatizer.lemmatize(token)
    lemma_tokeni.append(lemma)

print(lemma_tokeni)
```

```
['student', 'studying', 'different', 'subject', 'student', 'study', 'every', 'day', 'others', 'studied', 'exam']
```

VAŽNO

Koraci poput uklanjanja stop riječi, stemminga i lematizacije u NLP-u su analogni feature engineeringu u klasičnom machine learningu. Njihov cilj je pojednostaviti reprezentaciju podataka i smanjiti dimenzionalnost, ali pritom moramo paziti jer svaka transformacija može ukloniti i dio informacije iz teksta.

5. PROBLEMI S HRVATSKIM JEZIKOM

NLP alati su najrazvijeniji za engleski jer postoji najviše podataka i istraživanja.

Za manje jezike:

- manje modela
- manji datasetovi
- više lingvistike

```
hr_corpus = "Studenti su učili obradu prirodnog jezika, a najviše vole tokene!"
```

```
tokens = word_tokenize(hr_corpus)
```

```
print (tokens)
```

```
['Studenti', 'su', 'učili', 'obradu', 'prirodnog', 'jezika', ',', 'a', 'najviše', 'vole', 'tokene', '!']
```

```
hr_tokeni = []
```

```
for token in tokens:
```

```
    token = token.lower()
```

```
    if token.isalnum():
```

```
        hr_tokeni.append(token)
```

```
print(hr_tokeni)
```

```
['studenti', 'su', 'učili', 'obradu', 'prirodnog', 'jezika', 'a', 'najviše', 'vole', 'tokene']
```

Počnite pisati kôd ili [generirati](#) uz pomoć umjetne inteligencije.

```
hr_tokeni_stop = []
```

```
for token in hr_tokeni:
```

```
    if token not in engleski_stop_words:
```

```
        hr_tokeni_stop.append(token)
```

```
print (hr_tokeni_stop)
```

```
['studenti', 'su', 'učili', 'obradu', 'prirodnog', 'jezika', 'najviše', 'vole', 'tokene']
```

Počnite pisati kôd ili [generirati](#) uz pomoć umjetne inteligencije.

Micanje stop riječi nije savršeno (riječ "su" nije maknuta).

ZADATAK

Napravite listu s hrvatskim stop riječima

```
stemmer = PorterStemmer()
```

```
stem_hr_tokeni = []
```

```
for token in hr_tokeni_stop:
```

```
    stem = stemmer.stem(token)
```

```
    stem_hr_tokeni.append(stem)
```

```
print(stem_hr_tokeni)
```

```
['studenti', 'su', 'učili', 'obradu', 'prirodnog', 'jezika', 'najviš', 'vole', 'token']
```

Steming ne radi. Hrvatski jezik se bazira na totalno drugačijim jezičnim pravilima

```
lemmatizer = WordNetLemmatizer()
```

```
hr_lemma_tokeni = []

for token in hr_tokeni_stop:
    lemma = lemmatizer.lemmatize(token)
    hr_lemma_tokeni.append(lemma)

print(hr_lemma_tokeni)

['studenti', 'su', 'učili', 'obradu', 'prirodnog', 'jezika', 'najviše', 'vole', 'tokene']
```

Lematizacija isto ne radi!

GRUPNI RAD

Pronaći rješenja problema micanja stop riječi, stemming i lematizaciju za HR jezik

Predstaviti ih, ukratko

Pokazati primjere

Odabrati glasnogovornika koji će predstaviti rezultate

✓ I ZA KRAJ

Danas smo naučili kako pripremiti tekst za analizu. Sljedeći korak je još važniji: kako taj tekst pretvoriti u brojeve koje model može razumjeti.

Počnite pisati kôd ili generirati uz pomoć umjetne inteligencije.